A PROJECT SYNOPSIS ON

"BIOMETRIC AND GEOFENCING BASED ATTENDENCE SYSTEM"

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING
IN COMPUTER ENGINEERING

Submitted by

Sanket Biradar – Roll No. CO4009
Sujay Bonde – Roll No. CO4011
Dnyaneshwar Danene – Roll No. CO4013
Aditya Jagtap – Roll No. CO4027

Under the guidance of

Dr. D.M Yadav, Principal

DEPARTMENT OF COMPUTER ENGINEERING

PK Technical Campus, Chakan

Savitribai Phule Pune University

2026–27

Abstract

Conventional attendance-marking methods used in colleges, such as roll calls, signature registers, RFID cards, and barcode cards, have a major limitation: they verify a token or signature rather than the actual identity of the student. This makes proxy attendance easy to perform and difficult to detect. To address this problem, this project proposes a Smart College Attendance System that uses three independent verification mechanisms: GPS-based geofencing, face verification, and liveness detection. Geofencing confirms that the student is physically present near the classroom, face verification confirms the student's identity, and liveness detection ensures that the presented face belongs to a live person rather than a photograph or replayed video. Attendance is recorded only when all three verification checks are successfully completed, and the result is immediately reflected on the teacher's dashboard.

The system follows a three-service architecture. The Java Spring Boot backend manages authentication, user and attendance management, business rules, database operations, geofence validation, and real-time updates using WebSocket. A Python FastAPI service handles computer-vision tasks such as face detection, face embedding generation, face matching, and liveness verification. The React with TypeScript frontend provides a unified interface for students and teachers to enroll users, start attendance sessions, mark attendance, and view real-time and historical attendance records. PostgreSQL is used for persistent data storage and enforces database-level uniqueness to prevent duplicate attendance entries for the same student and session, including during concurrent requests. The proposed system aims to make college attendance more secure, reliable, automated, and resistant to proxy attendance, while providing immediate and transparent attendance information to teachers.

1. Introduction

Marking attendance is a routine task that, at institutional scale, consumes real class time and produces records that are tedious to compile and easy to falsify. A roll call or a signature register can always be gamed by a friend answering or signing on someone's behalf, and an RFID card is just as easy to lend out. What is actually needed is a way of confirming that the specific enrolled student is both present in the right place and physically there in front of the camera at that moment — not merely that a card, a signature, or a name was submitted.

This project, the "Smart College Attendance System," addresses that need by combining three independent signals before accepting an attendance mark: the student's live GPS location must fall within a teacher-defined radius of the classroom (geofencing), the face captured from the student's device camera must match their previously enrolled face (face recognition), and the capture must be shown to come from a live person rather than a static photo or a replayed video (liveness detection). Only when a request clears all three checks does the backend write an attendance record and broadcast it to the teacher's screen in real time over WebSocket, so a class of students being marked present is visible to the teacher as it happens rather than after the fact.

1.1 Background

Face recognition and liveness detection have matured to the point where reasonably accurate results are achievable with open libraries such as OpenCV and MediaPipe, running comfortably on ordinary laptop or server hardware without specialised camera or lighting equipment. At the same time, browsers now expose the Geolocation API directly, making GPS-based verification straightforward to add to a purely web-based system with no extra hardware at all. This project treats "business logic, database access, and security" and "computer-vision inference" as two genuinely different kinds of work and assigns them to two different backend services accordingly: a Java Spring Boot service for the former, and a Python FastAPI service — reachable only internally, never by the browser — for the latter. A React and TypeScript frontend sits on top of both, and PostgreSQL persists everything the Java service needs to enforce identity, enrollment, sessions, and attendance history.

2. Problem Statement

1. Manual Attendance
   Regular classroom attendance through roll calls and registers is time-consuming and requires continuous teacher involvement.

2. Proxy Attendance
   Roll numbers, signatures, ID cards, RFID cards, etc. can be misused by students to mark attendance on behalf of others.

3. Online Attendance
   Online attendance systems may allow students to mark attendance from outside the classroom without reliable physical presence verification.

4. Lack of Identity Verification
   Traditional systems verify a card, ID, or login credentials rather than confirming the actual identity of the student.

5. Location and Photo Spoofing
   Location-only systems can be manipulated, while face systems using only photographs can be fooled using static images or screenshots.

6. Lack of Liveness Verification
   Without liveness detection, recorded videos or replayed facial images may be used to bypass face verification.

7. Need for Smart Solution
   Therefore, a system is required that supports both regular and online attendance and combines geofencing, face verification, and liveness detection to reduce proxy attendance and provide automatic, secure, and real-time attendance records.

3. Objectives

1. To develop a smart college attendance system for secure and automated attendance management.
2. To verify student presence using GPS-based geofencing, face recognition, and liveness detection.
3. To implement face enrollment and face verification for accurate identification of students.
4. To implement liveness detection to prevent attendance through photographs, screenshots, or replayed videos.
5. To provide separate interfaces for students, teachers, and administrators for efficient attendance management.
6. To provide real-time attendance updates to teachers through the system dashboard.
7. To generate and export attendance reports in CSV and Excel formats along with attendance analytics.
8. To prevent duplicate attendance entries and maintain secure, reliable, and consistent attendance records in the database.
9. To support both regular classroom and online attendance while reducing the possibility of proxy attendance.
10. To provide a scalable and user-friendly attendance platform that reduces manual effort and improves attendance monitoring.

4. Literature Survey

Various attendance systems have been developed to reduce the problem of proxy attendance, but each approach has its own limitations. Traditional methods such as roll calls and attendance registers are simple to use, but they are time-consuming and depend heavily on manual verification. RFID and barcode-based systems make attendance faster, but they verify the card or ID rather than the actual student, so sharing a card can still result in proxy attendance. Fingerprint-based systems provide better identity verification, but they require dedicated biometric sensors and additional hardware, which may increase the cost and maintenance requirements of the system.

Face recognition-based attendance systems provide a more convenient alternative because they can use a standard camera to identify students. However, face recognition alone cannot determine whether the person is physically present or whether a photograph or recorded video is being used. Liveness detection can improve security by checking whether the captured face belongs to a live person, while GPS-based geofencing can verify whether the student is present within the required classroom area. Therefore, combining face verification, liveness detection, and geofencing provides a more reliable approach than using any single method alone. Based on these observations, the proposed system integrates all three verification methods to reduce proxy attendance and provide secure attendance marking. The system also uses a separate computer-vision service for face-related processing, while the main backend manages authentication, attendance rules, and other business operations, making the overall system more organized and modular.

5. Proposed System and Methodology

The system is organised as three cooperating layers. A React and TypeScript frontend is the only interface students and teachers use, and it talks exclusively to a Java Spring Boot backend over HTTPS and WebSocket. The Spring Boot backend owns authentication, course and enrollment data, attendance sessions, the geofence calculation, orchestration of the verification sequence, and all database access through PostgreSQL. A Python FastAPI service, reachable only from the Spring Boot backend over an internal Docker network and never exposed to the browser or the public internet, performs all computer-vision work: face detection, embedding generation, face matching, and the liveness challenge. This separation keeps sensitive model inference isolated from any direct client request, and keeps the browser's trust relationship limited to a single, auditable backend.

5.1 Attendance-Marking Workflow

1. A teacher creates a course and enrolls students, then creates an attendance session with a classroom location, a geofence radius, and a start/end time window.
2. A student, once logged in, sees the active session on their dashboard and initiates the "Mark Attendance" flow.
3. The browser's Geolocation API captures the student's current latitude, longitude, and accuracy, which is sent to the Java backend.
4. The Java backend computes the Haversine distance between the student's location and the session's centre point, and rejects the attempt if it falls outside the configured radius.
5. On a successful geofence check, the Java backend requests a liveness challenge from the Python service (for example, "blink" or "turn your head left"), which is displayed to the student with a time limit.
6. The student completes the requested action on camera; captured frames are sent through the Java backend to the Python service, which verifies the challenge was genuinely completed live.
7. On a successful liveness check, the student's face is captured and sent through the Java backend to the Python service, which detects the face, generates an embedding, and compares it against the student's previously enrolled embedding.
8. If the face match confidence clears the configured threshold, the Java backend inserts an attendance record — protected by a database-level uniqueness constraint on (student, session) — and marks the student present.
9. The Java backend immediately broadcasts the new attendance record over WebSocket to a per-session topic, and the teacher's Live Attendance dashboard updates without a manual refresh.
10. If any of the three checks fails, the attempt is rejected with a specific reason (out of radius, liveness failed, face mismatch, and so on) and no attendance record is created.

6. System Architecture

Figure 1 (to be inserted as a block/layer diagram in the full report) shows the browser communicating only with the Spring Boot API over HTTPS and a STOMP/SockJS WebSocket channel. Spring Boot owns authentication and role-based access control, course and enrollment management, attendance sessions, the geofence calculation, the WebSocket broadcaster, report generation, and all orchestration of calls into the Python service. The Python FastAPI service sits behind Spring Boot only — in the deployment configuration it has no port published to the host or the public network, so only Spring Boot's internal service address can reach it. PostgreSQL is accessed exclusively by the Spring Boot service; the Python service never talks to the database directly, and only ever receives and returns data (images, embeddings, verification results) through the Java layer.

This boundary — "business logic, database access, and security" in Java, and "computer-vision inference" in Python, both wrapped in a React interface — keeps the system's most sensitive components (the database and the biometric matching logic) behind a single, auditable gateway, and keeps the two backend teams able to develop and test independently once the internal API contract between them is agreed and frozen.

7. Technology Stack

Layer	Technology	Purpose
Frontend	React, TypeScript, Tailwind CSS	Student- and teacher-facing web interface, camera and geolocation capture
Charts (interactive)	Recharts / Chart.js	Live analytics dashboards inside the React app
Backend (public)	Java Spring Boot	Auth, courses, sessions, geofencing, orchestration, WebSocket, reports
Backend (internal, CV)	Python FastAPI	Face detection/embedding/matching and liveness challenge, called only by Java
Database	PostgreSQL	Users, courses, sessions, attendance, face embeddings, audit logs
Authentication	Spring Security + JWT	Stateless token-based auth with role-based authorization
Computer vision	OpenCV, MediaPipe	Face detection, quality checks, embedding generation and comparison
Liveness detection	MediaPipe face-mesh (blink / head-turn)	Anti-spoofing challenge verified before face match
Geolocation	Browser Geolocation API + Haversine formula	Client-side capture, server-side distance verification
Analytics (static reports)	Matplotlib	Server-rendered chart images for downloadable reports
Real-time updates	WebSocket (STOMP over SockJS)	Live attendance broadcast to the teacher dashboard
Reports	Apache POI (Excel), CSV export	Session and course attendance reports
Deployment	Docker, Docker Compose	Containerised deployment of all four services

8. Database Design (Overview)

The system uses PostgreSQL as the main database, and all database operations are handled through the Java Spring Boot backend. The database contains the following main tables:

1. Users: Stores common user information such as login details and roles.
2. Students: Stores student-specific information.
3. Teachers: Stores teacher-specific information.
4. Courses: Stores details of courses or subjects offered by the college.
5. Enrollments: Maintains the relationship between students and their enrolled courses.
6. Face Enrollments: Stores the face embedding associated with each student for face verification. The original enrollment photograph is not stored.
7. Attendance Sessions: Stores information about each attendance session, including the course, location, geofence radius, and session time.
8. Attendance: Stores the final attendance record, including marking time, distance from the session location, face-match score, and liveness result.
9. Verification Attempts: Maintains a record of verification attempts, including successful and failed geofence, face, and liveness checks along with the reason for failure.
10. Audit Logs: Records important system activities such as session creation, attendance-related actions, and report exports for security and traceability.
11. Duplicate Prevention: A database-level UNIQUE constraint on student_id and session_id ensures that the same student cannot be marked present more than once for the same session, even when multiple requests occur at the same time.

This database design provides secure, consistent, and organized storage for student information, attendance records, verification data, and system activities.

9. Advantages

- Verifies the actual person rather than a token, since a face and a live-liveness check cannot be handed to someone else the way a card or a signed name can.
- Adds a location check on top of identity verification, so a student cannot mark attendance from outside the classroom even if their face and liveness checks would otherwise pass.
- Liveness detection specifically closes the photo/video-replay gap that pure face-recognition systems are vulnerable to.
- Real-time WebSocket updates let a teacher see attendance being marked live, without waiting for a manual report.
- The computer-vision service is isolated on an internal-only network path, reducing the attack surface exposed to the public internet.
- A database-level uniqueness constraint prevents duplicate attendance even under concurrent or malicious repeated requests.
- No dedicated hardware (fingerprint sensors, RFID readers) is required — the system runs on the camera, GPS, and browser already present on students' and teachers' own devices.
- Exportable CSV/Excel reports and interactive dashboards reduce the manual effort of compiling attendance records.

10. Applications

- Lecture and lab attendance tracking in colleges and universities.
- Seminar, workshop, and event attendance verification where identity and physical presence both matter.
- Examination hall identity verification alongside physical seating checks.
- Any access-controlled space where confirming both the person's identity and their physical location is required, such as restricted labs or research facilities.

11. Future Scope

- A native mobile application to replace the browser-based camera and geolocation flow with a smoother in-app experience.
- Push notifications to parents or managers based on attendance status or absentee patterns.
- Automated, scheduled generation of periodic (weekly/monthly) attendance and leave reports.
- Stronger anti-spoofing measures — for example, depth-based or challenge-response liveness detection — to defend against more sophisticated deepfake or replay attacks, which the current blink/head-turn approach does not fully address.
- Integration with existing college ERP or student-management systems via REST APIs.
- Encrypted-at-rest storage of face embeddings using a managed key service, for institutions requiring stronger biometric-data protection than the current prototype provides.

12. Conclusion

- The system combines geofencing, face recognition, and liveness detection to provide secure and reliable attendance.
- The Java Spring Boot, Python FastAPI, and React components are separated to make the system modular and easier to manage.
- Database-level duplicate prevention and internal communication with the computer-vision service improve system security and reliability.
- The system makes proxy attendance more difficult than traditional registers, ID cards, or single-factor face recognition.
- Real-time attendance visibility helps teachers and administrators monitor student presence efficiently.

13. References

1. Spring Framework Documentation, "Spring Boot Reference Documentation," VMware/Spring Team. https://spring.io/projects/spring-boot
2. FastAPI Documentation, "FastAPI Framework, high performance, easy to learn." https://fastapi.tiangolo.com
3. Google, "MediaPipe Solutions," Face Mesh and Face Detection documentation. https://developers.google.com/mediapipe
4. OpenCV Team, "OpenCV Documentation." https://docs.opencv.org
5. PostgreSQL Global Development Group, "PostgreSQL Documentation." https://www.postgresql.org/docs
6. R. Jain, A. Sharma, "IoT Based Smart Attendance System Using Fingerprint Sensor," International Journal of Engineering Research & Technology (IJERT).
7. S. Kumar et al., "A Review on Biometric Based Attendance Systems," International Journal of Computer Applications.